Make My Marriage

API Design Document

REST API • Spring Boot • PostgreSQL • Marriage as Tenant

Purpose

Define the V1 REST API contract for the Make My Marriage responsive web application. Organizer APIs are scoped to a marriage tenant. Guest RSVP and photo upload use narrowly scoped, expiring tokens.

Baseline

Base path /api/v1 • JSON • UUIDs • ISO-8601 dates • JWT bearer auth • role authorization • consistent errors.

Prepared October 2026

1. API principles

Build a modular monolith with Spring Boot, React/TypeScript and PostgreSQL. Use Flyway migrations. Store photo bytes in private object storage; store metadata and moderation state in PostgreSQL.

Tenant rule: every organizer route selects a marriage via marriageId, but the backend must verify active membership and role. Every tenant-owned query includes marriage_id. A client-supplied ID is never proof of access.

Convention	Rule
IDs	UUID strings; avoid sequential IDs.
Time	ISO-8601 with offset for instants; store as TIMESTAMPTZ. Calendar-only dates use YYYY-MM-DD.
Pagination	?page=0&size;=20&sort;=startsAt,asc; response contains content, page, size, totalElements, totalPages.
Money	Explicit currencyCode; PostgreSQL NUMERIC. Calculate outstanding amount on server.
Validation	DTOs + Bean Validation; do not bind request JSON directly to entities.
Errors	Use a consistent error body; RFC 9457 Problem Details is a good baseline.
Versioning	Keep /api/v1 backward compatible; breaking changes require a new version.

2. Authentication and roles

Role	Permissions
ADMIN	Manage settings, membership, all modules and photo moderation.
MANAGER	Manage planning data according to permissions; restrict security-sensitive membership changes.
GUEST	No organizer API access. Only scoped RSVP/upload token actions.

Authentication endpoints

Method + path	Purpose
POST /auth/register	Create organizer account (public).
POST /auth/login	Authenticate and issue access token (public).
POST /auth/refresh	Rotate refresh token if implemented.
POST /auth/logout	Revoke session/refresh token if implemented.
GET /me	Return current user profile.
GET /me/marriages	List active marriage memberships.

Example login response

```
{
  "accessToken": "<short-lived-jwt>",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "user": {"id": "2f7f8d5e-4dd5-4b77-9ad0-4d99fd83ab12",
           "displayName": "Asha", "email": "asha@example.com"}
}
```

Use HTTPS in deployed environments. Hash passwords with a supported modern password encoder. If browser refresh uses cookies, use Secure + HttpOnly + appropriate SameSite settings and protect cookie-based flows against CSRF. Never log passwords, tokens or Authorization headers.

3. Marriage and member APIs

Method + path	Purpose	Access
POST /marriages	Create tenant and ADMIN membership in one transaction.	Authenticated
GET /marriages/{marriageId}	Read tenant details.	Active member
PATCH /marriages/{marriageId}	Update name, date, timezone, status.	ADMIN
GET /marriages/{marriageId}/members	List members.	ADMIN/MANAGER per policy
POST /marriages/{marriageId}/members/invitations	Invite organizer/manager.	ADMIN
PATCH /marriages/{marriageId}/members/{userId}	Change role/status.	ADMIN
DELETE /marriages/{marriageId}/members/{userId}	Remove member; protect last ADMIN.	ADMIN

Create marriage request

```
POST /api/v1/marriages
{
  "name": "Asha & Arjun Wedding",
  "weddingDate": "2027-02-14",
  "timezone": "Asia/Kolkata",
  "currencyCode": "INR",
  "themeKey": "TRADITIONAL_GOLD"
}
```

Return 201 Created and a Location header. Server generates id and creates ADMIN membership for the current user; never accept createdBy from the client.

4. Main module endpoints

Module	Routes / behavior
Events	GET/POST /marriages/{id}/events; GET/PATCH/DELETE /marriages/{id}/events/{eventId}. Validate start/end times.
Tasks	GET/POST /marriages/{id}/tasks; PATCH/DELETE /marriages/{id}/tasks/{taskId}. Checklist, due date, status, optional event/assignee.
Guests	GET/POST /marriages/{id}/guests; GET/PATCH/DELETE /marriages/{id}/guests/{guestId}. Paginate and protect personal data.
Invitations	POST /marriages/{id}/invitations; POST /marriages/{id}/invitations/{invitationId}/send. Support safe retries.
RSVP	GET/POST /public/rsvp/{token}. Token grants access only to its linked RSVP scope.
Expenses	GET/POST /marriages/{id}/expenses; GET/PATCH/DELETE /marriages/{id}/expenses/{expenseId}. Validate amounts.
Vendors	GET /vendors?category=&city;=&q;=; GET/POST /marriages/{id}/saved-vendors; PATCH/DELETE /marriages/{id}/saved-vendors/{id}. Catalog may be global; shortlist is tenant-scoped.
Albums	GET/POST /marriages/{id}/albums; PATCH/DELETE /marriages/{id}/albums/{albumId}.
Photos	GET /marriages/{id}/albums/{albumId}/photos; POST /marriages/{id}/albums/{albumId}/upload-links; POST /public/photo-upload/{token}.
Moderation	POST /marriages/{id}/photos/{photoId}/approve; POST /marriages/{id}/photos/{photoId}/reject. Require moderation permission.
Settings	GET/PATCH /marriages/{id}/settings. Allow-list theme, timezone, currency and keys.
Dashboard	GET /marriages/{id}/dashboard. Return scoped event, RSVP, expense and pending-photo summaries.

5. Request and response examples

List events

```
GET /api/v1/marriages/{marriageId}/events?page=0&size=20&sort=startsAt,asc
Authorization: Bearer <access-token>

{
 "content":[{
   "id":"1dd9d2c8-7c2b-4b87-84a7-30d9d6f0ab33",
   "marriageId":"7c7e0a8d-2a2f-4c0b-a5a0-41d1a3a5b701",
   "name":"Reception",
   "startsAt":"2027-02-14T18:30:00+05:30",
   "endsAt":"2027-02-14T22:00:00+05:30",
   "venueName":"Example Hall",
   "livestreamUrl":null
 }],
 "page":0,"size":20,"totalElements":1,"totalPages":1
}
```

Create expense

```
POST /api/v1/marriages/{marriageId}/expenses
{
 "category":"VENUE",
 "description":"Reception hall advance",
 "amount":75000.00,
 "amountPaid":25000.00,
 "currencyCode":"INR",
 "dueDate":"2027-01-10"
}

201 Created
{
 "id":"d92c4b2b-1e2b-4a80-9ac3-b3a2b1cc4f4f",
 "amount":75000.00,
 "amountPaid":25000.00,
 "outstandingAmount":50000.00,
 "currencyCode":"INR"
}
```

outstandingAmount is calculated server-side as amount - amountPaid; ignore any client-supplied value.

Public RSVP token

```
POST /api/v1/public/rsvp/{token}
{"response":"ACCEPTED","partySize":2,"dietaryResponse":"VEGETARIAN"}

200 OK
{"response":"ACCEPTED","partySize":2,
 "respondedAt":"2026-11-01T11:20:00+05:30"}
```

Do not return organizer notes, other household responses, internal IDs, or any data outside the token's scope.

6. Standard errors

```
{
 "type":"https://api.example.com/errors/validation",
 "title":"Validation failed",
 "status":400,
 "detail":"One or more fields are invalid.",
 "instance":"/api/v1/marriages/{marriageId}/events",
 "code":"VALIDATION_ERROR",
 "fieldErrors":[{"field":"name","message":"must not be blank"}],
 "traceId":"f1b7a3..."
}
```

HTTP	Meaning
200	Successful read/update/action with body.
201	Created; include Location header where practical.
204	Deleted successfully, no body.
400	Malformed JSON or invalid fields/query.
401	Missing, invalid or expired authentication/token.
403	Authenticated but permission denied; 404 may be used to conceal cross-tenant resources.
404	Resource missing or not visible in authorized tenant.
409	Duplicate or state conflict, such as duplicate membership.
413 / 415	Upload too large / unsupported media type.
429	Rate limit exceeded.
500	Unexpected failure; expose trace ID only, never stack trace/secrets.

7. Validation and business rules

- Verify all parent-child relationships within the same marriage.
- Event end time must not precede start time.
- Expense amounts are non-negative; amountPaid normally cannot exceed amount.

• Allow-list RSVP responses, statuses, theme keys, timezones, currencies and sort fields.
• Public tokens are random, hashed at rest, scoped, expiring, revocable and rate-limited.
• Validate upload size, MIME type and file signature; photos remain private/pending until approved.
• Use idempotency keys for retry-prone actions such as sending invitations where duplicate effects matter.

8. Security and privacy

• Require HTTPS; configure CORS for the intended frontend origin only.
• Apply Spring Security and explicit tenant membership/role checks.
• Test cross-marriage read, update, delete and nested-resource access.
• Rate-limit login, invitation, RSVP and upload endpoints.
• Redact secrets and personal data from logs; keep configuration in environment/secret storage.
• Validate livestream URLs; avoid server-side fetching of arbitrary user URLs.
• Audit member changes, expense edits and photo moderation.
• Re-check authorization after membership removal; avoid long-lived cached permissions.

9. OpenAPI / Swagger and Java implementation

Maintain an OpenAPI 3 specification and use springdoc-openapi for interactive documentation. Restrict or disable Swagger UI in production if appropriate. Document auth, roles, pagination, error responses and public token scopes.

Suggested dependencies: Spring Web, Spring Validation, Spring Security, Spring Data JPA, PostgreSQL driver, Flyway, springdoc-openapi, JUnit 5 and Spring Boot Test. Keep DTOs and service logic separated from JPA entities.

```
com.makemymarriage
  auth/             AuthController, AuthService, JwtService
  tenancy/          MarriageController, TenantAuthorizationService
  events/           EventController, EventService
  guests/           GuestController, InvitationService, RsvpService
  expenses/         ExpenseController, ExpenseService
  vendors/           VendorController, SavedVendorService
  photos/            PhotoController, PhotoModerationService
  settings/          MarriageSettingsController
  common/            ApiExceptionHandler, security config, DTOs
```

10. Implementation order and Definition of Done

Phase	Deliverable
1	Register/login, marriages, members and tenant authorization.
2	Events/tasks CRUD, validation, pagination and isolation tests.
3	Guest CRUD, invitations, secure RSVP tokens.
4	Expenses and vendor directory/shortlist.
5	Albums, private uploads, moderation and theme settings.
6	Dashboard, OpenAPI, audit logs, rate limits, deployment and end-to-end tests.

Definition of Done

Each endpoint has DTO validation, documented authorization, consistent errors and tests. All tenant-owned queries are scoped. Cross-marriage tests pass. Tokens expire/revoke correctly. Upload limits and moderation are enforced. OpenAPI documentation matches the implementation.

References

OpenAPI: https://spec.openapis.org/oas/latest.html
Spring Security: https://docs.spring.io/spring-security/reference/index.html
Spring Boot validation: https://docs.spring.io/spring-boot/reference/io/validation.html
RFC 9457 Problem Details: https://www.rfc-editor.org/rfc/rfc9457.html